REP-025 · Balance and pacing audit: how Fruit Market is won, lost and paced

Game	Fruit Market (rules version 2, `main` at 15f358a)
Report date	2026-10-27 03:36 (Asia/Kuala_Lumpur)
Type	Audit only. **No code changed.** Planning and documentation work, not a game change
Method	Read the rules engine line by line (`src/engine/game.ts` , `data.ts` , `ai.ts` , `daily.ts` , `rng.ts`), the mode set-up (`src/ui/App.tsx` , `src/ui/progress.ts` , `server/room.ts` , `server/meta.ts` , `src/net/protocol.ts`) and the scoring screens; then measured with computer-only simulations (`npm run balance` at 300 games per test, plus 4 extra read-only probes, 300–400 games each)
Approval status	Not applicable (findings and proposals; any change needs a plan and your approval)

Caution on the numbers. Every measured figure comes from computer players playing each other. People price, buy and take risks differently, so treat win rates as the shape of the game, not as human results. Human results need the playtest (DEC-016) and the telemetry proposed in section 6.

A. How the game is won and lost (overview)

- **You win by having the most cash when the last day's sales end.** Nothing else counts: unsold fruit, upgrades and favour cards are worth $0 at the end (`ranking()` in `src/engine/game.ts`).
- **Nobody is ever knocked out.** There is no bankruptcy, no health, no timer and no early finish. Every game lasts exactly its length: 8, 10 or 14 days. "Losing" just means not finishing first.
- **Ties:** most cash wins; if level, most fruit sold; if still level, the **seat order decides silently** (seat 0, which is you in solo, daily and league games). The results screen then wrongly says it was "decided on fruit sold".
- **Where money comes from:** buying fruit cheaply at the farmers market and selling it to customers through **secret, sealed prices** where the **cheapest offer sells first**. Freshness limits the top price you may ask and breaks price ties. Everything else (upgrades, traits, favour cards, events) moves the margins.
- **What really decides games** (measured): pricing skill (Hard beats Normal 84% of the time head to head), buying only what customers want, the Signboard and Cool Room upgrades, and the event deck. Turn order is a deliberate catch-up: the poorest shop buys first.
- **Long-term goals outside a single game:** the 6-stage Market League (win to advance, unlimited retries), the daily challenge (score = final cash on a shared leaderboard), and 12 achievements.

B. Win / loss condition table

#	Outcome	Exact check (code)	When it's evaluated	Notes and problems
1	**Game ends**	`finishReveal()`: `if (s.day >= s.config.days) phase = 'over'`	After the final day's sales reveal	No evening on the last day: no upgrades, no ageing, no night fines
2	**Win (1st place)**	`ranking()`: sort by `cash` desc, then `stats.sold` desc	Phase `over`	Third tie-break is the array's original order (stable sort) = seat order. **Hidden outcome**
3	2nd–4th place ("loss")	Same ranking	Phase `over`	No other penalty
4	Dead heat message	`Results.tsx`: `margin === 0` → "A dead heat, decided on fruit sold!"	Results screen	**Wrong text** when sold counts are also equal (then seat order decided). Measured: 0.33% of 3–4-player games
5	No defeat / elimination	none exists	—	Cash can't go below 0 (buys need cash; the inspector fine is capped at your cash)
6	"Broke" state (soft loss)	Not in code; arises when cash < $1 and no ripe stock	Any morning	You can still play but can't buy (every fruit costs ≥ $1). Only favour cards (Free Cash, Free Fruit) can rescue you. Measured: 0.3–0.6% of computer players in 3-player games, 2.2% in 4-player Hard games, typically from day 4–6
7	Online leaver	Server: grace period, then a computer plays the seat (`aiActFor`)	Live	The match always finishes; the winner is still decided by cash
8	Local quit	Leaving a local game	Any time	No result; only `quit_day` telemetry
9	Daily challenge score	`server/meta.ts`: board sorted by `cash`, then `sold`, then submit time; the server replays your moves to verify	On submit	Your "win" against the 3 rivals is separate from your leaderboard rank
10	League stage cleared	`recordLeague()`: `ranking(g)[0] === me` and you're on that stage	Results screen	Lose = replay the same stage (new shuffle). Clearing stage 6 unlocks the League Champion achievement and the Royal colour
11	Achievements (optional)	`checkAchievements()` in `src/ui/progress.ts`	Results screen	`rich` ($200) reached by 0–1.8% of computer players and effectively impossible in 4-player Hard games (best seen $185). `comeback` needs being last on day 5

Order inside a day (it matters for every outcome above): morning (event revealed; delivery added) → buying in turn order → customers arrive → everyone seals prices → sales resolved customer by customer (yesterday's waiting customers first, then today's) → evening upgrades (not on the last day) → night: inspector fine, then everything ages, then the next day starts.

Conflicting or unreachable outcomes found

- On the **final day**, Health Inspector and Heatwave's extra ageing do nothing (they act at night, and there is no last night). Priority can't be played (no evening). These cards still appear as the final day's event.
- The **seat-order tie-break** favours seat 0 (you in solo, daily and league games, the room host online).
- `rich` ($200) is practically unreachable in 4-player games.

C. Balance values (grouped by system, with code references)

Legend for "Type": **C** = constant in `src/engine/data.ts` (one place, easy to tune); **H** = hard-coded number inside logic; **F** = formula; **T** = also written in player-facing text (must be edited too). ⚠ marks a value that needs edits in several places. ★ marks the highest-impact controls.

C1. Run length, start and end

Value	Current	Changes	Affects	Where	Type
Game length ⚠	8 / 10 / 14 days (default 10); league 8 or 10; daily 10	Chosen at set-up	Everything; final cash roughly scales with days (3-player Hard: $61 / $81 / $127)	`App.tsx` `setDays`, options list; `server/room.ts` allowed list; `src/net/protocol.ts` `RoomConfig`, `DEFAULT_CONFIG`; `progress.ts` `LEAGUE`; `daily.ts`	H ×5, T (`RULES.md`, `GAME-BRIEF.md`)
Starting cash ⚠	$10	Nest Egg makes it $16	Day 1–2 buying power, upgrade timing	`START_CASH` (data.ts); `applyTrait()` `START_CASH + 6`	C, H, T (`RULES.md`, `HowTo.tsx` "$10", trait text "$16 instead of $10")
Winner rule ★	most cash → most sold → seat order	Never	The goal	`ranking()` game.ts	H
Final-day stock value	$0	Never	End-of-game buying	implicit	—

C2. Supply: the farmers market

Value	Current	Changes	Affects	Where	Type
Delivery size ★	each card fruit: `max(1, round(base × scale))`; fruits not on the card: base 2	Daily	How much fruit exists; price pressure	`startDay()`; `FARMER_CARDS` (20 cards, 3 fruits each, 4–6 per fruit)	C + F
Supply scale ★	`buyCapacity × (1 + 0.5 ×`	Rises over the game;	≈100% of total buy capacity	`supplyScale()` game.ts; 17.15 = `FARMER_AVG_QTY`	F with H 1, 0.5

Value	Current	Changes	Affects	Where	Type
	`progress) ÷ 17.15`	rises when anyone builds a Handcart or has Big Basket	on day 1 → 150% on the last day		
Unripe deliveries	9 of 20 farmer cards; mango (2 nights) or watermelon (1 night); only if "Unripe fruit" is on	Per card	The best margins in the game (unripe mango $2 → sells as fresh)	`FARMER_CARDS[].unripe`, `FRUIT.*.unripeSlots`	C
Market fruit ages	1 step a night (+1 Heatwave)	Nightly	Cheap old fruit available every day	`nightfall()`	H
Buy cost by freshness	blueberry 2/1/1 (fresh/okay/not so), unripe 1; watermelon 3/2/1, 1; apple 4/3/1, 1; mango 4/2/1, unripe 2; floor $1	Events ±$1	Margins	`FRUIT.*.cost`, `marketCost()`	C, T (`RULES.md` table)
Buy limit ⚠	4 a day; 5 with Handcart; +1 Big Basket	Upgrade or trait	Throughput, **and everyone's supply and demand** (see finding E7)	`BASE_BUY`, `CART_BUY`, `buyLimit()`	C, T (upgrade text, `RULES.md`, `HowTo.tsx`)
Shelf size ⚠	8; 14 with Bigger Shelf	Upgrade	Rarely binding	`BASE_SHELF`, `BIG_SHELF`	C, T

C3. Demand: customers

Value	Current	Changes	Affects	Where	Type
Demand target ★	`buyCapacity × (0.40 + 0.45 × progress)` units/day, split over (1 + walk-ins) customers	Daily	How much can be sold at all; price wars	`demandScale()` game.ts	F with H 0.4, 0.45
Card size	24 customers, 3–6 fruit each (avg 4.375), 1–3 fruit types	Deck reshuffles when empty	Order sizes	`RAW_CUSTOMERS`; `CARD_AVG_QTY`	C
Order rounding	`max(1, round(qty × scale))`	Per draw	Small early games get more demand than the target (every want is at least 1)	`drawCustomer()`	H

Value	Current	Changes	Affects	Where	Type
Walk-ins ⚠	1 a day; 2 from the halfway day (day 6 of 10, day 5 of 8, day 8 of 14); events ±1	At halfway	More, smaller orders → more showdowns in the second half	`baseWalkIns()` (threshold 0.5)	H
Forecast customer	1, known a day early	Daily	Planning	`customersArrive()`	H
Waiting customers	Unserved new customers wait 1 more day and are served first; then leave (no penalty)	Daily	Carry-over demand	`finishReveal()`, `resolve()`	H
Customer top price ★	card `max` + fruit shift (apple −1, mango +2), + event; weighted averages: blueberry $6.5, watermelon $7.4, apple $7.7, mango $9.1	Events	Revenue ceiling	`RAW_CUSTOMERS[].max`, `PRICE_SHIFT`, `effectiveMax()`	C
Event extra customers	Influencer, Gym Club, Hotel Chef	With their event	One-day demand spike	`EVENT_CARDS[].extra`	C (not price-shifted, unlike normal cards)

C4. Fruit life and selling

Value	Current	Changes	Affects	Where	Type
Life tracks ★	blueberry 3 slots (not so, okay, fresh); watermelon 5 (+1 unripe); apple 7; mango 4 (+2 unripe)	Fruit moves one slot a night	Spoilage risk	`FRUIT.*.ripe`, `unripeSlots`	C, T
Freshness discount ★	blueberry −4/−2/0; watermelon −3/−1/−1/0/0; apple −2/−1/−1/−1/0/0/0; mango −3/−1/0/0	With age	Top price allowed per fruit	`FRUIT.*.drop`, `capAt()`	C, T
Price floor	$1; Minimum Price event: watermelon 5, apple 5, mango 6, blueberry 4	Event	Price wars	`minPrice()`	H, C
Who sells first	lowest price → fresher fruit → tie edge (Favoured Trade card, or Haggler v richer shop) → buying order (poorest first)	Every sale	All income	`resolve()` sort	H
Customers pay	your sealed price (not their top price)	—	Undercutting	`resolve()`	H
Signboard tip ⚠	+$1 per fruit sold	Upgrade	Income	`resolve()`; display repeated in `animator.tsx`	H ×2, T

C5. Turn order and catch-up

Value	Current	Where	Type
Buying order ★	Day 1 random; then **poorest first** (ties round the table); Priority card jumps the queue	`startDay()`	H
Tie-break by buying order	Poorer shop wins full price-and-freshness ties	`resolve()`	H

C6. Events (13 cards, one a day from day 2, tomorrow's shown a day early; reshuffled when used)

Event	Effect	Measured income vs same-day average (2p / 4p)
Festival	+$3 all prices, +1 walk-in	+11.1 / +8.1
Superfood	+$3 blueberry, Gym Club wants 4 at $8	+10.6 / +4.5
Mango Craze	+$3 mango, Influencer wants 3 at $11	+10.4 / +4.2 (largest gap between shops: 25–35)
Hotel Order	Hotel Chef orders 7	+6.6 / +1.0
Bad Harvest	delivery ×0.5, +$2 prices	+1.7 / +2.6
Heatwave	+$4 watermelon, everything ages 2 steps tonight	−1.1 / −0.1
Minimum Price	price floors	−2.3 / +1.1
Good Harvest	+3 each fruit, costs −$1	−3.6 / −2.0
Health Inspector	$2 fine per not-so fruit at night (capped by cash)	−3.9 / −2.0
Price Hike	costs +$1	−4.0 / −2.3
Lorry Breakdown	no delivery	−6.0 / −2.9
Rainy Day	no walk-in, costs −$1	−8.0 / −4.8
Slow Season	−$2 all prices, no walk-in	**−12.0** / −7.3

Where: `EVENT_CARDS` (data.ts, C), applied in `startDay()`, `marketCost()`, `priceMod()`, `customersArrive()`, `nightfall()`; text repeated in `RULES.md`.

C7. Upgrades (each evening each shop is offered 3 it hasn't built; it may build any of those it can afford)

Upgrade	Cost	Effect (code)	Measured: AI that never builds it (3p, control 37% win)	Payback estimate
Signboard ★	$8	+$1 per fruit sold	**26.3%** (biggest loss)	~2.5 days (2p, 3.2 sales/day) to ~3.2 days (4p, 2.5/day)

Upgrade	Cost	Effect (code)	Measured: AI that never builds it (3p, control 37% win)	Payback estimate
Cool Room	$7	ages only on even-numbered nights (`s.day % 2 === 1` skips); immune to Heatwave	32.3%	Indirect: keeps fruit fresh (higher top price, wins freshness ties)
Bigger Shelf	$3	8 → 14 shelf	29.7%	Only when hoarding
Handcart	$6	4 → 5 buys a day	32.0%	Low: shops are limited by customers, not by buying
Shop Bell	$4	2 favour cards now; hand 3 → 4	34.3%	~2 cards × ~$2–3 value
Juice Press	$4	up to 3 spoiled fruit a night → $2 each	**37.3% (no loss at all)**	Never for careful play: computers spoil 0.1–0.7 fruit per game

Rushing any single upgrade on the first evening loses (13–26% win). Building all six loses badly (–$87 versus rivals). No upgrades at all: 21% (vs 37% for the computer's normal choices). Where: `UPGRADES` (C, T), effects in `buyLimit`, `shelfSize`, `handLimit`, `resolve`, `ageShop`, `buyUpgrade` (H). The Juice numbers (3, $2) are hard-coded in `ageShop()`.

C8. Merchant traits (pick 1 of 3 on day 1; computers take a random one)

Trait	Effect	Where	Measured win rate (3p, fair 33.3%)
Bargain Hunter	first fruit each day $1 cheaper (min $1)	`buyCost()`	**42.0%** (+$7.7 vs rivals)
Charmer	3 favour cards at start; hand +1	`applyTrait()`, `handLimit()`	39.0%
Nest Egg	start with $16	`applyTrait()`	38.0%
Local Favourite	+$3 per customer you complete alone (no daily limit)	`resolve()`	35.3%
Haggler	wins tied prices against richer shops	`tieEdge()`	30.7%
Big Basket	+1 buy a day	`buyLimit()`	**29.0%**

C9. Favour cards (12-card deck; earned when you alone complete a customer; at most 1 per shop per day; hand limit 3)

Free Cash +$3 · Free Fruit (any market fruit, free, ignores the buy limit) · Priority (buy first tomorrow) · Good Riddance (sell all ripe stock back for $1 each; it returns to the market) · Favoured Trade (win tied prices today) · Fridge (all your ripe fruit one step fresher). Where: `FAVOURS`, `FAVOUR_DECK` (C), `playFavour()` (H: the $3, the $1), `resolve()` (earning rule).

C10. Computer difficulty

Knob	Easy	Normal	Hard	Where
Buy if value − cost ≥	0.3 (+ random ±2)	0.8	0.3	`pickPurchase()`
Pricing	rival's cap minus 0–1 (random)	best response, then −0 or −1	best response, then −1; reads tomorrow's event	`planBids()`, `HARD` constant
Keeps unsold value	—	30%	10%	`planBids()`
Upgrades	skips 60% of nights	list order with day limits, keeps ≥ $6	same	`pickUpgrade()`
Measured head to head (2p)	—	beats Easy 72%	beats Normal 84%, Easy 91%	`npm run balance`

C11. Modes

Mode	Set-up	Where
Daily	You vs Normal, **Hard**, Normal; 10 days; same decks for everyone	`daily.ts`
League	1: 1 Easy (8 days) · 2: 2 Easy (8) · 3: 1 Normal · 4: 2 Normal · 5: Hard + Normal · 6: 3 Hard (all 10 days)	`LEAGUE` in `progress.ts`
Online	2–4 people, optional computers, 8/10/14 days, toggles	`server/room.ts`

Versioning rule: any change to C1–C10 that alters results needs `RULES_VERSION` (`src/net/protocol.ts`) raised. That **discards games saved in progress** (`store.ts`), keeps old and new apps out of each other's online rooms, and separates daily scores. League and mode set-ups (C11) can change without it.

D. Progression and pacing map

Units: money in $, time in game days. Computer-only measurements, 10-day games unless stated.

Early game: days 1–3 ("set-up")

- Day 1 is thin: one forecast customer plus one walk-in, orders scaled to 40% of buying capacity. Average income per shop on day 1: **$6.3 (2p), $4.9 (4p)**, against $10 starting cash. Most shops spend nearly all their cash on day 1.
- Supply is about **2.5× new demand on day 1** (formula: supply 100% vs demand 40% of capacity). Unbought market fruit ages and becomes cheap, so not-so-fresh fruit at $1 is almost always available.
- First upgrades: computers buy the Signboard on day ~2.6, Handcart ~2.5–3.1, Cool Room ~2.8, Bell ~2.8–3.4. The AI only builds when it keeps ≥ $6 afterwards, so the $8 Signboard needs $14 in hand.
- Traits make the biggest difference here: Nest Egg's +$6 is about one full day's income at this point.

Mid game: days 4–6

- Income climbs: 2p **22–27*aday* * *,4*p* * *13–16 a day**.
- Walk-ins double at the halfway day (day 6 of 10), so there are more but smaller orders, more showdowns and longer reveals.
- Juice Press is usually built here (day ~5) and pays nothing back for careful players.
- **Who's ahead on day 5 usually wins:** 66% (2p), 54% (3p), 47% (4p), against fair chances of 50/33/25%.

Late game: days 7–10

- Income plateaus: 2p **28–30*aday* * *,4*p* * *16–19 a day** (demand reaches 85% of capacity; supply 150%).
- Upgrades stop (the computer needs 3–6 days left for each). Last day: no evening; unsold fruit is worth $0.
- Computers still buy a full basket (~4) on the last day to sell the same day.

Whole run

Measure	2 players	3 players	4 players	Source
Average final cash (Hard mirror)	$131	$81	$56	balance, probes
Final cash range (10th–90th percentile)	$92–170	$49–117	$24–91	probes
Median winning margin	$27	$17	$15	balance
Close games (≤ $3)	8%	10%	12%	balance
Blowouts (≥ $20)	63%	45%	41%	balance
Lead changes per game	2.7	3.6	4.2	balance
Comeback from last on day 5	33%	19%	9%	balance
Sales at ≤ $2 ("price war")	6%	—	16%	balance
Sales near the customer's top price	51%	—	20%	balance
Fruit spoiled per shop per game	0.1	—	0.7	balance

Profit per fruit bought (3p): mango $2.59 > blueberry $2.21 > watermelon $1.85 > apple $1.58. In 4-player games every fruit falls to $1.1–1.8.

Real time per run: not measured. From the animation code, each showdown takes about 2.3–3.2 seconds (0.55× with "Fast"); people's thinking time dominates and isn't recorded. Assuming 30–90 seconds per day, a 10-day game takes about 5–15 minutes. Telemetry should measure this (section 6).

League difficulty curve (computer standing in for you, 300 games per stage):

Stage	Easy-level player	Normal-level player	Hard-level player
1 Village Square (1 Easy, 8 days)	47%	65%	87%
2 Harbour Steps (2 Easy, 8 days)	27%	67%	84%
3 Old Town (1 Normal)	29%	54%	84%
4 Spice Lane (2 Normal)	7%	38%	80%

Stage	Easy-level player	Normal-level player	Hard-level player
5 Grand Bazaar (Hard + Normal)	5%	**11%** (~9 tries)	54%
6 Royal Market (3 Hard, 4 players)	5%	**5%** (~20 tries)	28%
Daily challenge (Normal, Hard, Normal)	2%	**8%**	41%

Downtime, spikes and snowballing

- Downtime: day 1 (little to sell) and long showdown sequences in the late game.
- **Difficulty spike:** league stages 4 → 5 → 6 for an average player (38% → 11% → 5%).
- **Snowball:** the day-5 leader wins well above fair, and blowouts are common. The poorest-first buying rule does help: the old rotating order had 62% blowouts in 3-player games against 45% now. But being poor enough to buy first on 4 or more days still means winning only 15.5% of the time.
- **Unrecoverable:** a shop with less than $1 and no ripe fruit can't buy again without a lucky favour card.
- **Player count changes the game more than any setting:** 4-player games earn less than half what 2-player games earn.

E. Prioritised findings and proposed adjustments

Confirmed = shown by the code or by measurement. **Hypothesis** = needs testing or human playtest.

#	Finding	Evidence	Player impact	Where to change	Proposal and side effects
E1	**League cliff at stages 5–6** (confirmed)	Normal-level player: 38% → 11% → 5%	Average players stall for 10–20 tries; the "League Champion" goal feels out of reach	`LEAGUE` in `src/ui/progress.ts` (no rules change, no `RULES_VERSION` bump)	Add a step between (e.g. Hard + Normal + Normal), or make stage 6 three players. Test until each step loses no more than ~10–15 points for a Normal-level player. Side effect: easier champion achievement
E2	**4-player economy is squeezed** (confirmed)	Final cash $56 vs $131; 16% of sales at ≤ $2; only 20% near top price	More price wars and fewer good days; `rich` achievement out of reach; Royal Market feels grim	`demandScale()` and `supplyScale()` coefficients in `game.ts`; could depend on the number of shops	Hypothesis: lower supply growth (the 0.5) or raise demand (0.40/0.45) for 3–4 shops. Tune **together**: the gap between supply and demand drives prices. Side effect: changes daily scores (always 4 players), so needs `RULES_VERSION`

#	Finding	Evidence	Player impact	Where to change	Proposal and side effects
E3	**Snowballing and blowouts** (confirmed; how much is too much is a hypothesis)	Day-5 leader wins 54% (3p); blowouts 41–63%; close games 8–12%	Many games are decided by mid-game	`startDay()` order, `resolve()` tie rules, the event deck	Options to test one at a time: tie advantage for the poorest beyond full ties; smaller late-game event swings. Side effect: skill matters less
E4	**Juice Press is a dead choice** (confirmed)	AI without it wins as often (37.3% vs 37%); spoilage < 1 fruit per game	Wastes $4 and an evening for new players who pick it	`UPGRADES.juicer` and `ageShop()` (3 and $2 hard-coded)	Rework so it helps careful players too, e.g. also juices unsold fruit on the last day, or more per fruit. Needs testing so it doesn't become a farm (I tested "buy $1 dying fruit to juice": it **loses**, 17% win)
E5	**Trait spread 29–42%** (confirmed at 300 games; ±3 points noise)	Bargain Hunter 42%, Big Basket 29%, Haggler 30.7%	Trait pick quietly decides some games	`TRAITS` text + `buyCost()`, `buyLimit()`, `tieEdge()`	Haggler almost never fires because freshness breaks ties first (code order). Make it apply before freshness, or give it a small extra. Big Basket is weak because buying capacity isn't the limit: try +1 shelf space or a $1-off second fruit. Trim Bargain Hunter only if playtests agree. Test each trait **independently**
E6	**Handcart and Big Basket feed everyone** (confirmed in code)	`buyCapacity()` adds every shop's limit into demand and supply	Your Handcart also gives rivals more customers and fruit	`buyCapacity()`, `demandScale()`, `supplyScale()`	Decide if intended (the comment says it keeps the market tight). If not, base the scales on the shop count instead. Side effect: Handcart becomes stronger
E7	**Hidden seat tie-break and wrong text** (confirmed)	`ranking()` stable sort; Results text	0.3% of games; seat 0 wins dead heats	`ranking()`, `Results.tsx`	Add an explicit third rule (e.g. fewer fruit spoiled, or a shared win) and correct the text. Low effort
E8	**Final-day events that do nothing** (confirmed)	Inspector and Heatwave ageing act at night; no last night	A "dramatic" final event that changes nothing	`startDay()` event draw	Don't draw night-only events for the last day, or apply them at the final sales. Low effort
E9	**No way back from broke**	0.3–2.2% of computer players;	A player can sit out the rest	`canBuy()`, `startDay()`	Hypothesis: a tiny safety net (e.g. $1

#	Finding	Evidence	Player impact	Where to change	Proposal and side effects
	(confirmed, rare)	humans may be higher	of a game		morning credit when cash and ripe stock are both 0). Measure humans first (E-telemetry)
E10	**Slow Season and Rainy Day are big shared losses** (confirmed size; fun is a hypothesis)	−12/−8 a day in 2p	Can feel like a wasted day	`EVENT_CARDS`	Soften (e.g. Slow Season −1*instead of* −2) if playtesters dislike them
E11	**Daily challenge is hard for an average player** (confirmed for the computer stand-in)	Normal-level wins 8%; Hard rival in the line-up	Daily "wins" are rare, but leaderboard rank still rewards	`daily.ts` rival levels	Optional: Normal, Normal, Hard on alternate days. Changing it changes daily seeds' outcomes; do it at a date boundary
E12	**Rounding inflates small orders** (confirmed, minor)	`max(1, round(qty × scale))` at scale 0.46–0.7	2-player early days have slightly more demand than intended	`drawCustomer()`	Leave unless tuning E2
E13	**Numbers duplicated in text** (confirmed)	Section C ⚠ items	A balance change can leave the rules, how-to or upgrade text wrong	`RULES.md`, `HowTo.tsx`, `UPGRADES` / `TRAITS` text, `animator.tsx` tip	When tuning, update together; better: build those texts from the constants
—	Checked, **not** a problem	"Juice farming" (17% win, loses); "buy only the cheapest old fruit" (0% win); last-day buying (sold the same day, warned "Unsold = $0")	—	—	No change

F. Missing information and playtest questions

Not available from the code or simulations

- How people actually price (do they undercut by $1 like the Hard computer, or price high?).
- Real play time per day and per game, and where people stop (only `quit_day` exists).
- Human win rates per difficulty, trait and mode; how often humans go broke.

- Your target experience: how often should an average player win against Normal? Is a 4-player game meant to feel tighter? Should the League be finishable by most players?

Playtest questions (fit the DEC-016 kit)

1. After the first game: did you understand why you won or lost a sale? (freshness and ties)
2. Did any day feel pointless (day 1, Slow Season, a broke stall)?
3. Which upgrade did you build first and why? Did the Juice Press feel useful?
4. Which trait did you pick, and did it feel strong?
5. Did the game feel decided before the end? From which day?
6. How long did it take, and was that the right length?

---

6. Tuning and measurement plan

Order of testing (expected impact first)

1. **League steps (E1)**: no rules change, quick to simulate and ship.
2. **Player-count economy (E2)**: supply and demand coefficients, tuned **together** (their ratio matters), checked in 2, 3 and 4 players.
3. **Dead or weak choices (E4, E5, E6)**: Juice Press, Haggler, Big Basket, each **independently**, one change per test run.
4. **Snowballing (E3)** after the economy, because price wars and margins change it.
5. **Small fixes (E7, E8)**: tie-break and final-day events. Safe; ship with the next rules change.
6. **Difficulty and daily line-up (E11)** last, once the economy is settled.

Tune together: demand 0.40/0.45 with supply 1.0/0.5; customer top prices with fruit costs (margin); Signboard tip with upgrade cost. **Tune alone:** each trait, each upgrade, each event, the league line-up.

Telemetry to add (anonymous counters, in line with `docs/ANALYTICS.md`; adding events changes data handling, so it needs its own approved plan):

- Game result by mode, number of players, length and rival levels (win/lose, final rank, winning margin bucket).
- Trait picked × win; upgrades built (which, which day) × win.
- Fruit spoiled and unsold at the end (buckets); fruit sold; share of sales at the customer's top price.
- Rank on day 5 × final rank (snowball and comeback).
- Game duration in minutes (bucket) and minutes per day.
- Times a player was broke (cash < $1, no ripe stock).
- League: attempts per stage and clears; daily: rank against the three rivals.

Small test matrix | Axis | Values | |---|---| | Players | 2, 3, 4 | | Rivals | all Easy, all Normal, mixed, all Hard | | Length | 8, 10, 14 days | | Strategy for the test seat | computer default; no upgrades; Signboard first; each trait fixed; cautious buyer (only forecast demand) | | Toggles | events on/off; unripe on/off |

Run each cell 300–400 games with `npm run balance` (plus the probes used here), compare against this report's baseline, then confirm the top 2–3 changes with 3–5 human playtesters before any release.

Appendix: probes used for this report (read-only, not committed)

- `curves` : supply and demand per day from `demandScale()` / `supplyScale()` for 2–4 players.
- `probes` : juice farming; broke shops; ties; last-day buying; final cash spread.
- `ladder` : league stage and daily win rates for Easy-, Normal- and Hard-level stand-ins (300 games each).
- `bottom` : "buy only the oldest, cheapest fruit" strategy (400 games).